

Chapter 5: Multiprocessors (Thread-Level Parallelism)– Part 2

Introduction

What is a parallel or multiprocessor system?

Why parallel architecture?

Performance potential

Flynn classification

Communication models

Architectures

Centralized sharedmemory

Distributed sharedmemory

Parallel programming

Synchronization

Memory consistency models

Synchronization

Communication – Exchange data

Synchronization – Exchange data to order events

- Mutual exclusion or atomicity

- Event ordering or Producer/consumer

 - Point to Point

 - Flags

 - Global

 - Barriers

Mutual Exclusion

Example

Each processor needs to occasionally update a counter

Processor 1

Load reg1, Counter

$\text{reg1} = \text{reg1} + \text{tmp1}$

Store Counter, reg1

Processor 2

Load reg2, Counter

$\text{reg2} = \text{reg2} + \text{tmp2}$

Store Counter, reg2

Mutual Exclusion Primitives

Hardware instructions

Test&Set

Atomically tests for 0 and sets to 1

Unset is simply a store of 0

```
while (Test&Set(L) != 0) {;}
```

Critical Section

Unset(L)

Problem?

Mutual Exclusion Primitives – Alternative?

Test&Test&Set

Mutual Exclusion Primitives – Fetch&Add

```
Fetch&Add(var, data)
{ /* atomic action */
  temp = var
  var = temp + data
}
return temp
```

E.g., let $X = 57$

P1: $a = \text{Fetch\&Add}(X, 3)$

P2: $b = \text{Fetch\&Add}(X, 5)$

If P1 before P2, ?

If P2 before P1, ?

If P1, P2 concurrent ?

Point to Point Event Ordering

Example

Producer wants to indicate to consumer that data is ready

Processor 1

$A[1] = \dots$

$A[2] = \dots$

.

.

$A[n] = \dots$

Processor 2

$\dots = A[1]$

$\dots = A[2]$

.

.

$\dots = A[n]$

Global Event Ordering – Barriers

Example

All processors produce some data

Want to tell all processors that it is ready

In next phase, all processors consume data produced previously

Use barriers

Memory Consistency Model - Motivation

Example shared-memory program

Initially all locations = 0

Thread 1

Thread 2

```
store 1 → y  
load x
```

```
store 1 → x  
load y
```

Execution (only shared-memory operations)

```
store 1 → y  
load x  
store 1 → x  
load y  
(x=0, y=1)
```

```
store 1 → y  
store 1 → x  
load x  
load y  
(x=1, y=1)
```

```
store 1 → y  
store 1 → x  
load y  
load x  
(x=1, y=1)
```

```
store 1 → x  
load y  
store 1 → y  
load x  
(x=1, y=0)
```

```
store 1 → x  
store 1 → y  
load y  
load x  
(x=1, y=1)
```

```
store 1 → x  
store 1 → y  
load x  
load y  
(x=1, y=1)
```

Thread 1 loads x.

$x = ?$

Thread 2 loads y.

$y = ?$

Memory Consistency Model - Motivation

Example shared-memory program

Initially all locations = 0

Processor 1

Data = 23

Flag = 1

Processor 2

while (Flag != 1) {;

... = Data

Execution (only shared-memory operations)

Processor 1

Write, Data, 23

Write, Flag, 1

Processor 2

Read, Flag, 1

Read, Data, ____

Memory Consistency Model: Definition

Memory consistency model

Order in which memory operations will appear to execute

⇒ What value can a read return?

Affects ease-of-programming and performance

The Uniprocessor Model

Program text defines total order = *program order*

Uniprocessor model

Memory operations appear to execute one-at-a-time in program order

⇒ Read returns value of last write

BUT uniprocessor hardware

Overlap, reorder operations

Model maintained as long as

maintain control and data dependences

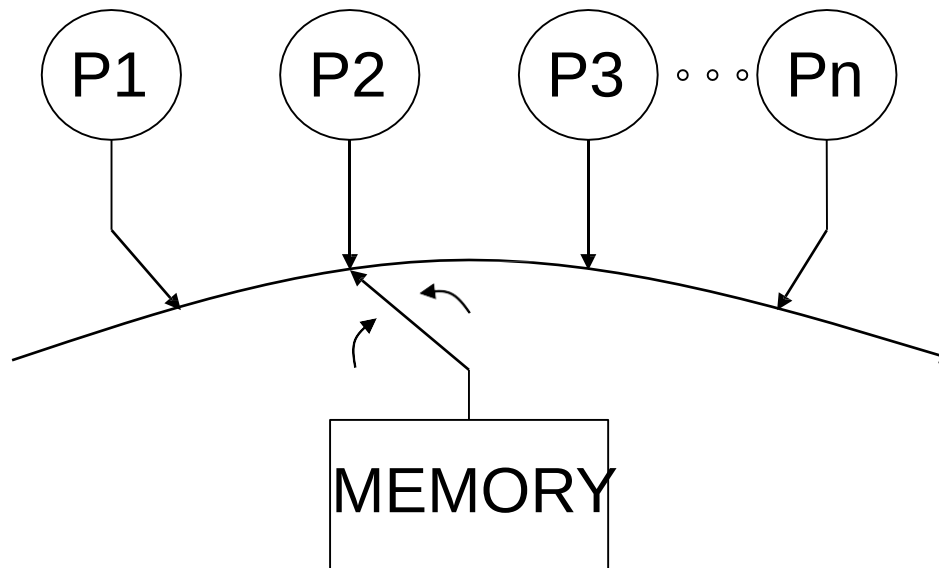
⇒ Easy to use + high performance

Implicit Memory Model

Sequential consistency (SC) [Lamport]

Result of an execution appears as if

- All operations executed in some **sequential order** (i.e., atomically)
- Memory operations of each process in **program order**



Understanding Program Order – Example 1

Initially Flag1 = Flag2 = 0

P1

Flag1 = 1

if (Flag2 == 0)

critical section

P2

Flag2 = 1

if (Flag1 == 0)

critical section

Execution:

P1

(Operation, Location, Value)

Write, Flag1, 1

Read, Flag2, 0

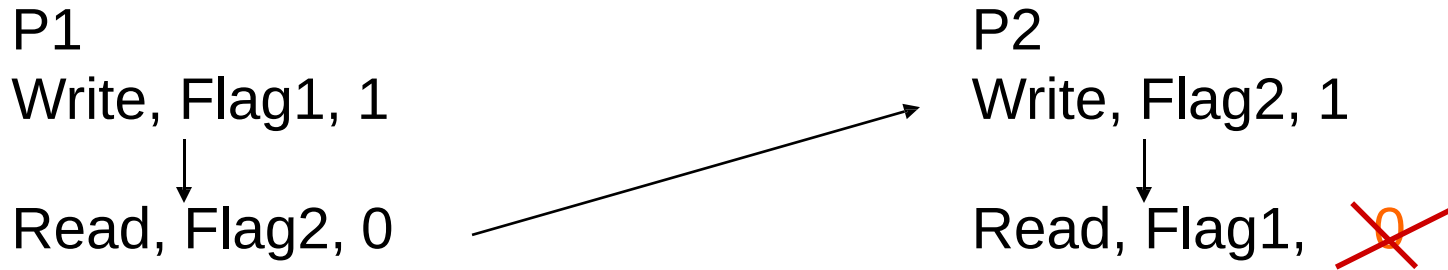
P2

(Operation, Location, Value)

Write, Flag2, 1

Read, Flag1, ____

Understanding Program Order – Example 1



Can happen if

- Write buffers with read bypassing
- Overlap, reorder write followed by read in h/w or compiler
- Allocate Flag1 or Flag2 in registers

Understanding Program Order - Example 2

Initially A = Flag = 0

P1

A = 23;

Flag = 1;

P1

Write, A, 23

Write, Flag, 1

P2

while (Flag != 1) {;

... = A;

P2

Read, Flag, 0

Read, Flag, 1

Read, A, _____

Understanding Program Order - Example 2

Initially A = Flag = 0

P1

A = 23;

Flag = 1;

P2

while (Flag != 1) {;

... = A;

P1

Write, A, 23

Write, Flag, 1

P2

Read, Flag, 0

Read, Flag, 1

Read, A, ~~0~~

Can happen if

Overlap or reorder writes or reads in hardware or compiler

Understanding Program Order - Example 3

Example:

Suppose we have a processor where a write miss takes 50 cycles to establish ownership, 10 cycles to issue each invalidate after ownership is established, and 80 cycles for an invalidate to complete and be acknowledged once it is issued. Assuming that four other processors share a cache block, how long does a write miss stall the writing processor if the processor is sequentially consistent? Assume that the invalidates must be explicitly acknowledged before the coherence controller knows they are completed.

Suppose we could continue executing after obtaining ownership for the write miss without waiting for the invalidates; how long would the write take?

Answer:

The total time for the write = **50 + 40 + 80 = 170** cycles.

In comparison, the ownership time is only **50** cycles. With appropriate write buffer implementations, it is even possible to continue before ownership is established.

Understanding Program Order: Summary

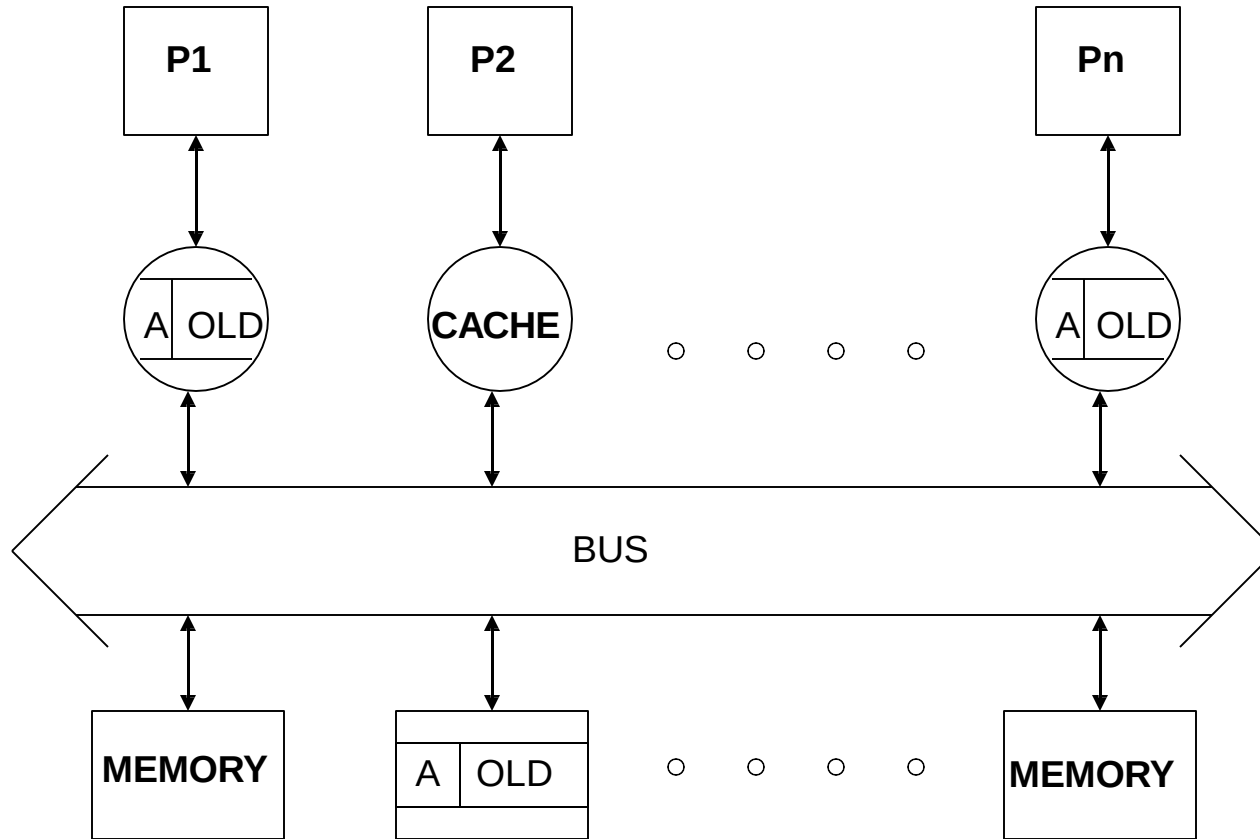
SC limits program order relaxation:

Write → Read

Write → Write

Read → Read, Write

Understanding Atomicity



A mechanism needed to propagate a write to other copies

⇒ Cache coherence protocol

Cache Coherence Protocols

How to propagate write?

Invalidate -- Remove old copies from other caches

Update -- Update old copies in other caches to new values

Understanding Atomicity - Example 1

Initially $A = B = C = 0$

P1

A = 1;

B = 1;

P2

A = 2;

C = 1;

P3

while (B != 1) {;

while (C != 1) {;

tmp1 = A;

P4

while (B != 1) {;

while (C != 1) {;

tmp2 = A;

Understanding Atomicity - Example 1

Initially $A = B = C = 0$

P1	P2	P3	P4
$A = 1;$	$A = 2;$	$\text{while } (B \neq 1) \{;$	$\text{while } (B \neq 1) \{;$
$B = 1;$	$C = 1;$	$\text{while } (C \neq 1) \{;$	$\text{while } (C \neq 1) \{;$
		$\text{tmp1} = A; \quad \text{1}$	$\text{tmp2} = A; \quad \text{2}$

Can happen if updates of A reach P3 and P4 in different order

Coherence protocol must serialize writes to same location

(Writes to same location should be seen in same order by all)

Understanding Atomicity - Example 2

Initially $A = B = 0$

P1

$A = 1$

P2

while ($A \neq 1$) ;
 $B = 1$;

P3

while ($B \neq 1$) ;
 $tmp = A$

P1

Write, A, 1

P2

Read, A, 1
Write, B, 1

P3

Read, B, 1

Read, A, ~~0~~

Can happen if read returns new value before all copies see it

SC Summary

SC limits

Program order relaxation:

Write → Read

Write → Write

Read → Read, Write

When a processor can read the value of a write

Unserialized writes to the same location

Alternative

- (1) Aggressive hardware techniques proposed to get SC w/o penalty using speculation and prefetching

But compilers still limited by SC

- (2) Give up sequential consistency
Use relaxed models

Classification for Relaxed Models

Typically described as system optimizations - **system-centric**

Optimizations

Program order relaxation:

Write → Read

Write → Write

Read → Read, Write

Read others' write early

Read own write early

All models provide safety net

All models maintain uniprocessor data and control dependences,
write serialization

Some System-Centric Models

Relaxation:	W $\rightarrow$ R Order	W $\rightarrow$ W Order	R $\rightarrow$ RW Order	Read Others' Write Early	Read Own Write Early	Safety Net
IBM 370	✓					serialization instructions
TSO	✓				✓	RMW
PC	✓			✓	✓	RMW
PSO	✓	✓			✓	RMW, STBAR
WO	✓	✓	✓		✓	synchronization
RCsc	✓	✓	✓		✓	release, acquire, nsync, RMW
RCpc	✓	✓	✓	✓	✓	release, acquire, nsync, RMW
Alpha	✓	✓	✓		✓	MB, WMB
RMO	✓	✓	✓		✓	various MEMBARs
PowerPC	✓	✓	✓	✓	✓	SYNC

System-Centric Models: Assessment

System-centric models provide higher performance than SC
BUT 3P criteria

Programmability?

Lost intuitive interface of SC

Portability?

Many different models

Performance?

Can we do better?

Need a higher level of abstraction

An Alternate Programmer-Centric View

One source of consensus

Programmers need SC to reason about programs

But SC not practical today

How about the next best thing...

A Programmer-Centric View

Specify memory model as a contract

System gives sequential consistency

IF programmer obeys certain rules

+ Programmability

+ Performance

+ Portability

The Data-Race-Free-0 Model: Motivation

Different operations have different semantics

P1

A = 23;

B = 37;

Flag = 1;

P2

while (Flag != 1) {;

... = B;

... = A;

Flag = Synchronization; A, B = Data

Can reorder data operations

Distinguish data and synchronization

Need to

- Characterize data / synchronization
- Prove characterization allows optimizations w/o violating SC

Data-Race-Free-0: Some Definitions

Two operations conflict if

- Access same location
- At least one is a write

Data-Race-Free-0: Some Definitions (Cont.)

(Consider SC executions $\Rightarrow$ global total order)

Two conflicting operations **race** if

- From different processors
- Execute one after another (consecutively)

P1

Write, A, 23

Write, B, 37

Write, Flag, 1

P2

Read, Flag, 0

Read, Flag, 1

Read, B,

Read, A,

Races usually “**synchronization**,” others “**data**”

Can optimize operations that *never race*

Data-Race-Free-0 (DRF0) Definition

Data-Race-Free-0 Program

All accesses distinguished as either **synchronization** or **data**

All **races** distinguished as **synchronization**

(in any SC execution)

Data-Race-Free-0 Model

Guarantees SC to data-race-free-0 programs

It is widely accepted that data races make programs hard to debug
independent of memory model (even with SC)

Distinguishing/Labeling Memory Operations

Need to distinguish/label operations at all levels

- High-level language
- Hardware

Compiler must translate language label to hardware label

Java: `volatile`, `synchronized`

C++: `atomic`

Hardware: fences inserted before/after synchronization

Data-Race-Free Summary

The idea

- Programmer writes data-race-free programs

- System gives SC

For programmer

- Reason with SC

- Enhanced portability

For hardware and compiler

- More flexibility

Finally, convergence on hardware and software sides

(BUT still many problems...)

Acknowledgements

These slides contain material developed and copyright by:

- Sarita Adve (UIUC)
- Nathan Beckmann (CMU)
- David Patterson (UC Berkeley)
- Yuanchun Zhang (NUDT)
- Qianni Deng (SJTU)